Verifiable monitors

Luko van der Maas

November 20, 2024

1 Preliminaries

Definition 1 *A deterministic finite automaton (DFA) $\mathcal{A}$ is a tuple $\mathcal{A} = (Q, \Sigma, \delta, \iota, F)$ where:*

- *Q is a finite set of states,*
- *Σ is an alphabet,*
- *$\delta \colon Q \times \Sigma \rightharpoonup Q$ is a (partial) transition function,*
- *ι is the initial state, and*
- *$F \subseteq Q$ is a set of accept (or: final) states.*

From [2]

Definition 2 *The language $\mathcal{L}(()\mathcal{A})$ of a DFA $\mathcal{A}$ is the set of all words accepted by the $\mathcal{A}$.*

Definition 3 *A (discrete-time) Markov chain (MC) is a tuple $\mathcal{M} = (S, \mathbf{P}, \iota, AP, L)$ where:*

- *S is a countable, nonempty set of states,*
- *$\mathbf{P} : S \times S \to [0, 1]$ is the transition probability function such that for all states s :*

$$\sum_{s' \in S} \mathbf{P}(s, s') = 1$$

- *ι is the initial state, and*
- *AP is a set of atomic propositions, and*
- *$L : S \to 2^{AP}$ is a labeling function.*

Additionally, observations can be added to an MC by adding a Set of observations O, together with an observation function $obs \colon S \to O$.

From [2]

Definition 4 *The probability measure $Pr^{\mathcal{M}}$ of an MC $\mathcal{M}$ is the unique probability measure following from the canonical σ-algebra associate with mc.*

Definition 5 *Given an MC $\mathcal{M} = (S, \mathbf{P}, \iota, AP, L)$, the paths conforming to a trace τ are defined as*

$$\text{Paths}(\tau) = \{\pi \in \Pi^{fin} \mid obs(\pi) = \tau\}$$

Definition 6 *Given an MC $\mathcal{M}$, the cylinder set $\text{Cyl}(\pi)$ of a finite path π is the set of all infinite extensions of π in $\mathcal{M}$.*

Definition 7 *A Markov decision process is a tuple $\mathcal{M} = (S, Act, \mathbf{P}, \iota, AP, L)$ where*

- *S is a countable set of states,*
- *Act is a set of actions,*
- *$\mathbf{P} : S \times Act \times S \to [0, 1]$ is the transition probability function such that for all states $s \in S$ and actions $\alpha \in Act$:*

$$\sum_{s' \in S} \mathbf{P}(s, \alpha, s') \in \{0, 1\}$$

- *ι is the initial state,*
- *AP is a set of atomic propositions, and*
- *$L : S \to 2^{AP}$ is a labeling function.*

From [2]

2 Problem Statements

We have two problems we want to solve. The no false alarms problem and the no missed alarms problem. They have slightly different problem statements and therefore also slightly different solutions.

2.1 No false alarms

All traces τ from the MC which are in the monitor have at least a probability $p \in I$ of reaching the good state.

Definition 8 *Given a Markov chain $\mathcal{M} = (S, \mathbf{P}, \iota, AP, L)$ with observations in the set of colors C, a specification DFA $\mathcal{A} = \left(Q_{\mathcal{A}}, 2^{AP}, \delta_{\mathcal{A}}, \iota_{\mathcal{A}}, F_{\mathcal{A}}\right)$, AP, and a DAG monitor DFA $\mathcal{B} = (Q_{\mathcal{B}}, C, \delta_{\mathcal{B}}, \iota_{\mathcal{B}}, F_{\mathcal{B}})$. For all traces τ from the MC and in the monitor, the probability of $\pi \in \mathcal{L}(\mathcal{A})$ given that π conforms to τ is in a given interval I*

$$\operatorname*{in}^{I}_{\tau \in \mathcal{L}(\mathcal{M}) \cap \mathcal{L}(\mathcal{B})} Pr^{\mathcal{M}}(\pi \in \mathcal{L}(\mathcal{A}) \mid \pi \in \text{Cyl}(\text{Paths}(\tau)))$$

2.2 No missed alarms

All traces τ which reach the good state with probability $p > t$, are in the monitor. By taking the contrapositive, we can instead show that all traces τ which are not in the monitor reach the good state with probability $p \leq t$.

Definition 9 *Given a Markov chain $\mathcal{M} = (S, \mathbf{P}, \iota, AP, L)$ with observations in the set of colors C, a specification DFA $\mathcal{A} = \left(Q_{\mathcal{A}}, 2^{AP}, \delta_{\mathcal{A}}, \iota_{\mathcal{A}}, F_{\mathcal{A}}\right)$, AP, and a DAG monitor DFA $\mathcal{B} = (Q_{\mathcal{B}}, C, \delta_{\mathcal{B}}, \iota_{\mathcal{B}}, F_{\mathcal{B}})$. For all traces τ from the MC and not in the monitor, the probability of $\pi \in \mathcal{L}(\mathcal{A})$ given that π conforms to τ is in a given interval I*

$$\underset{\tau \in \mathcal{L}(\mathcal{M}) \cap \overline{\mathcal{L}(\mathcal{B})}}{\text{in}^I} Pr^{\mathcal{M}}(\pi \in \mathcal{L}(\mathcal{A}) \mid \pi \in \text{Cyl}(\text{Paths}(\tau)))$$

3 Algorithm

Our algorithm will proceed in four steps. First, we take the MC product [2] of the MC $\mathcal{M}$ and the specification $\mathcal{A}$. Then, we will combine the product MC with the monitor into an MDP. Next we will add end transitions to this MDP for all states that are accepted by the underlying monitor. Lastly, we use Paynt [1] to model check the created MDP.

3.1 Applying the specification to the MC

We take the MC product as defined by Baier and Katoen [2] to create the product MC $\mathcal{M} \oplus \mathcal{A}$.

Intuition The created product MC is the original MC, but every good state is labeled.

3.2 Constructing the product MDP

To create the MDP, we first create its states. The states are the product of the states of the product MC and the monitor. These are denoted as $\langle s, q_{\mathcal{B}} \rangle$ for $s \in S_{\mathcal{M} \oplus \mathcal{A}}$, and $q_{\mathcal{B}} \in Q_{\mathcal{B}}$. As actions in the product MDP we take the observations C of the MC $\mathcal{M}$.

The transitions in the MDP are defined here in two steps. First we create a not-yet-normalized transition function P' on the states in the MDP. Next, we normalize the transition function by creating redirecting all remaining probability mass of an action in a state towards the initial state in P''.

$$P'(\langle s, q \rangle, c, \langle s', q' \rangle) = \begin{cases} P(s, s') & \text{if } obs(s') = c \wedge q' = \delta(q, c) \\ 0 & \text{otherwise} \end{cases}$$

$$P''(\langle s, q \rangle, c, \langle s', q' \rangle) = \begin{cases} 1 - \sum_{s'' \in S} \sum_{q'' \in Q} P'(\langle s, q \rangle, c, \langle s'', q'' \rangle) & \text{if } s' = \iota_{\mathcal{M} \oplus \mathcal{A}} \wedge q' = \iota_{\mathcal{B}} \\ P'(\langle s, q \rangle, c, \langle s', q' \rangle) & \text{otherwise} \end{cases}$$

Lastly, the atomic propositions are $\{\circledcirc_{\mathcal{A}}, \circledcirc_{\mathcal{B}}\}$, where accepting states in the product MC are labeled by $\circledcirc_{\mathcal{A}}$ and accepting states in the monitor are labeled by $\circledcirc_{\mathcal{B}}$.

Intuition The created MDP can be seen as the original product MC $\mathcal{M} \oplus \mathcal{A}$, where we can choose which observation to observe, but only if the monitor contains the transition labeled by the observation.

3.3 Adding end transitions

We add two new states to the MDP, the *goal* and *stop* states, each only having a self loop. Next, we will add an *end* action to every state with the label $\circledcirc_{\mathcal{B}}$. If the state also contains the $\circledcirc_{\mathcal{A}}$ label, we direct the *end* action to the *goal* state, otherwise we direct the *end* action to the stop state.

Intuition Since we are talking about finite traces in the monitor, we need to give the MDP a way to end a path. This is done by taking the *end* action. If we take the end action in a good state of the product MDP, we reach the *goal* state, if it was a bad state, then we reach the *stop* state.

3.4 Model checking using Paynt

We create a POMDP which will be model checked by Paynt [1] for reachability of the goal state. The observations in this POMDP are the states of the monitor. Using this POMDP we calculate

$$\mathbb{P}_{\max}[\diamond goal]$$

to find the maximum probability and

$$1 - \mathbb{P}_{\max}[\diamond stop]$$

to find the minimum probability. Finally, we can verify this upper and lower bound with the interval I.

Intuition In order to find the traces in the monitor which give the maximum and minimum probability of reaching a good state, we associate a trace τ with a scheduler σ_τ in an MDP. Such a scheduler is called consistent.

Definition 10 *A* consistent scheduler *is a scheduler which generates only one trace*

$$\exists_{\tau \in \Sigma_{obs}} \ \forall_{\pi \in Paths(\sigma)} \ obs(\pi) = \cdots \iota \tau e^{\star}$$

where ι is the observation of the initial state and e is the observation of the goal and stop state.

By model checking the created POMDP, we don't yet ensure that every scheduler is consistent, instead we only ensure semi-consistence.

Definition 11 *A semi-consistent scheduler σ is a scheduler which has the following property*

$$\exists_{\tau \in \Sigma_{obs}} \forall_{\pi \in Paths(\sigma)} \; obs(\pi) = (\iota\tau^{+})^{\star}$$

where τ^{+} can be a different prefix of τ for every loop in $()^{\star}$.

Thus, a semi-consistent scheduler does not have to take and end action. However, by maximizing the probability of reaching either the *goal* or *stop* state, we ensure that if the *end* action can be taken, it will be taken. And if a path takes the *end* action, almost surely any other path will also take the *end* action at some point. This property ensures that when maximizing semi-consistent schedulers we will always get a consistent scheduler, and thus a single trace.

4 Alternative formulations of the problem

Instead of giving a specification DFA, we could instead give a TL specification [3] which is evaluated for every state. Since TL has both future and past operators both now reachability, $\diamond\!\!\!\!\cdot\; l$, and soon reachability, $\bigcirc^{n} \diamond\!\!\!\!\cdot\, l$, for a label l can be expressed. While using a specification DFA can only express now reachability.

The only impact this reformulation of the problem would have is in step 1. Here, instead of taking the product MC, we check if the above described TL formula has a probability above a certain threshold. If that is the case we mark it with the $\circledcirc_{\mathcal{A}}$ label. This MC is then used for the subsequent steps.

References

[1] Roman Andriushchenko et al. "PAYNT: A Tool for Inductive Synthesis of Probabilistic Programs". In: *Computer Aided Verification*. Ed. by Alexandra Silva and K. Rustan M. Leino. Cham: Springer International Publishing, 2021, pp. 856–869. ISBN: 978-3-030-81685-8. DOI: 10.1007/978-3-030-81685-8_40.

[2] Christel Baier and Joost-Pieter Katoen. *Principles of Model Checking*. MIT Press, 2008. ISBN: 978-0-262-02649-9.

[3] Orna Lichtenstein, Amir Pnueli, and Lenore Zuck. "The Glory of the Past". In: *Logics of Programs*. Ed. by Rohit Parikh. Berlin, Heidelberg: Springer, 1985, pp. 196–218. ISBN: 978-3-540-39527-0. DOI: 10.1007/3-540-15648-8_16.